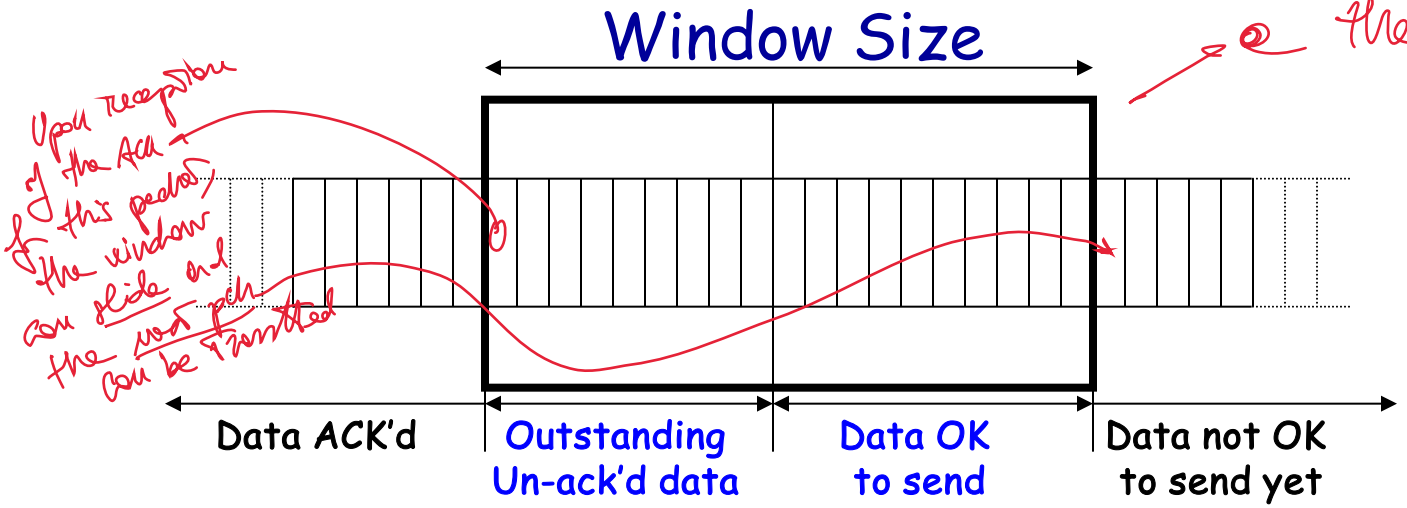
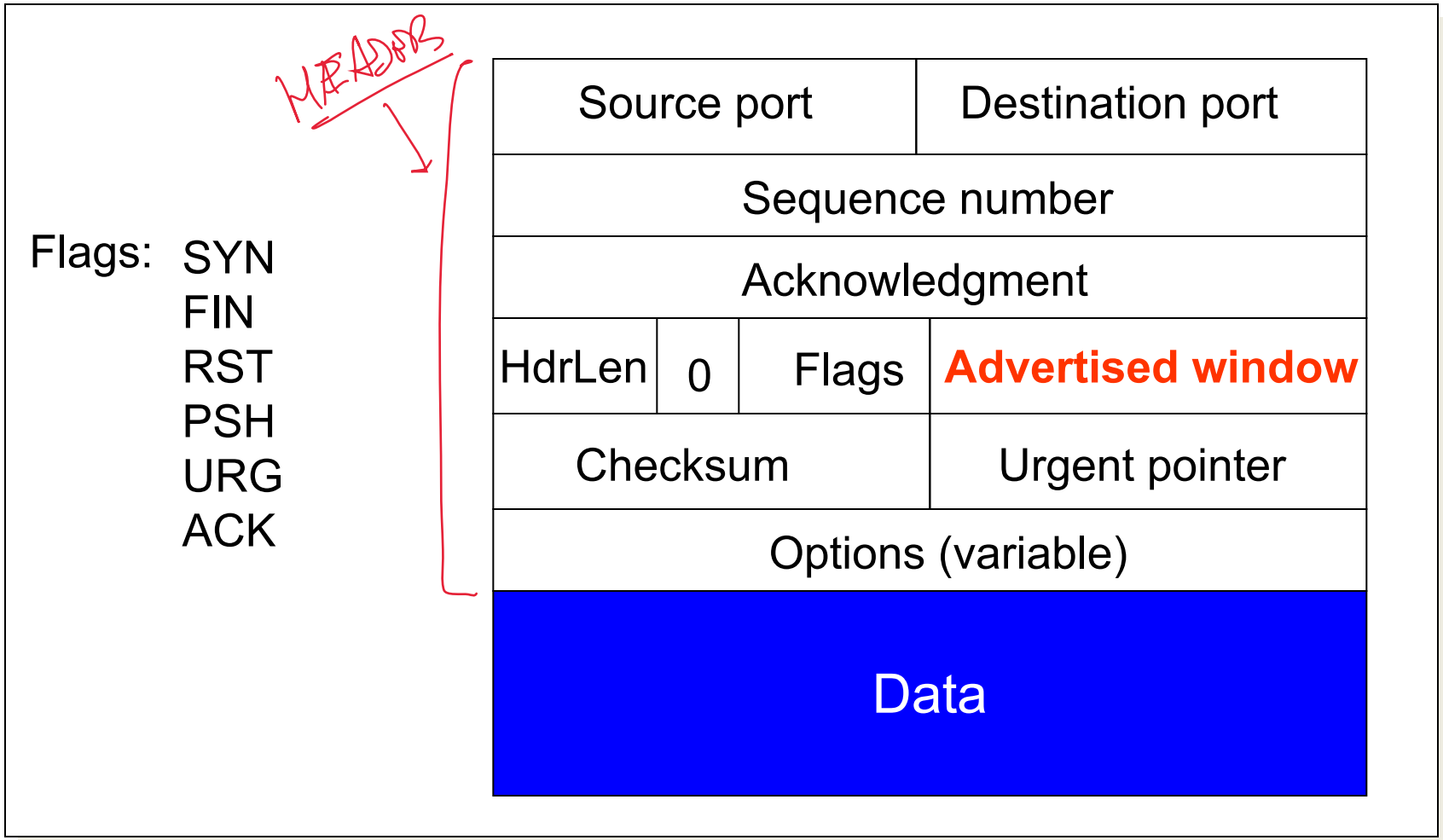
Receiver Buffering

- Window size
  - Amount that can be sent without acknowledgment
  - Receiver needs to be able to store this amount of data
- Receiver advertises the window to the receiver
  - Tells the receiver the amount of free space left
  - … and the sender agrees not to exceed this amount



TCP Header for Receiver Buffering

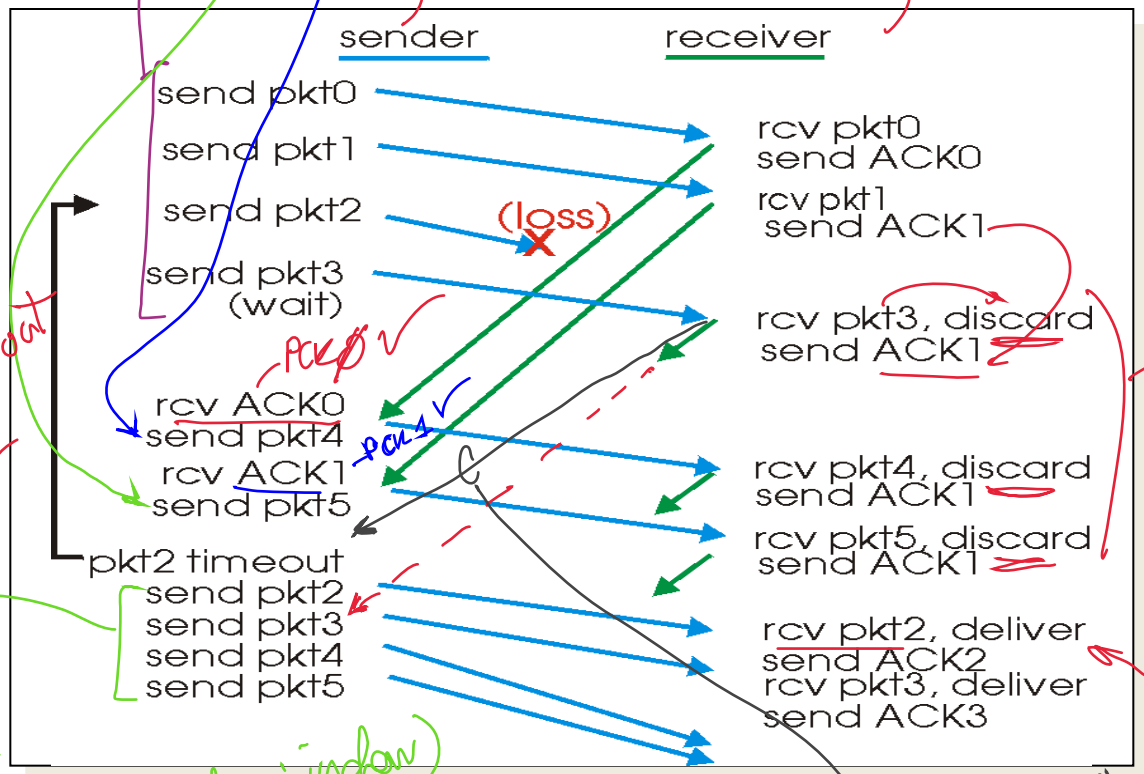




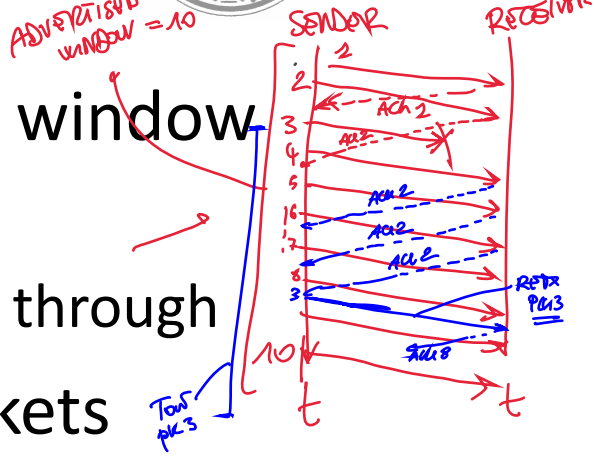
Fast Retransmission

Timeout is Inefficient

- Timeout-based retransmission
  - Sender transmits a packet and waits until timer expires
  - … and then retransmits from the lost packet onward

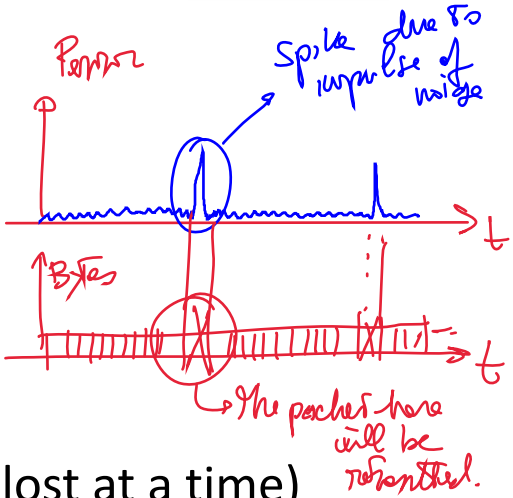


Fast Retransmission



- Better solution possible under sliding window
  - Although packet n might have been lost
  - … packets n+1, n+2, and so on might get through
- Idea: have the receiver send ACK packets
  - ACK says that receiver is still awaiting n^{th} packet
    - And *repeated* ACKs suggest later packets have arrived
  - Sender can view the "duplicate ACKs" as an early hint
    - … that the nth packet must have been lost
    - … and perform the retransmission early
- Fast retransmission
  - Sender retransmits data after the triple duplicate ACK

Effectiveness of Fast Retransmit

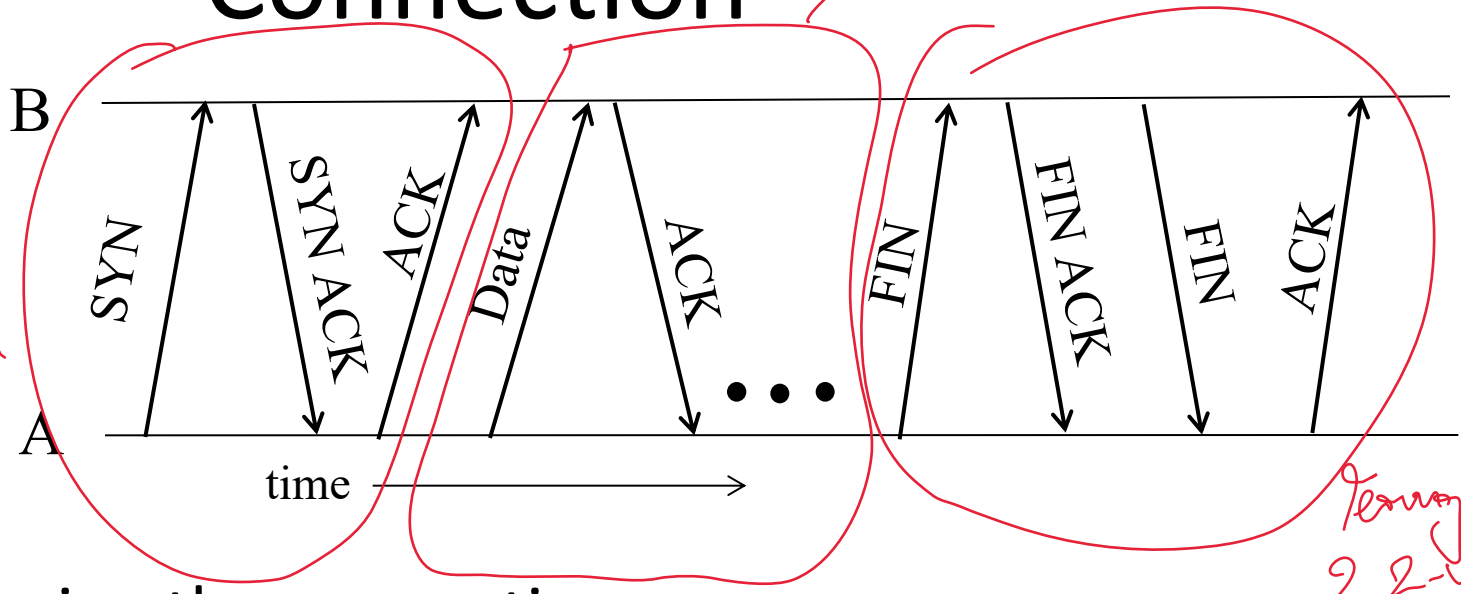


- When does Fast Retransmit work best?
  - Long data transfers
    - High likelihood of many packets in flight
  - Large window size
    - Support for many packets in flight
  - Low burstiness in packet losses (only one pck lost at a time)
    - Higher likelihood that later packets arrive successfully
- Implications for Web traffic
  - Most Web transfers are short (e.g., 10 packets)
    - Short HTML files or small images
  - So, often there aren't many packets in flight
  - … making fast retransmit less likely to "kick in"
  - Forcing users to like "reload" more often… ☺



Tearing Down the Connection

Tearing Down the Connection



- Closing the connection
  - Finish (FIN) to close and receive remaining bytes
  - And other host sends a FIN ACK to acknowledge
  - Reset (RST) to close and not receive remaining bytes

Sending/Receiving the FIN Packet



- Sending a FIN: close()
  - Process is done sending data via the socket
  - Process invokes “close()” to close the socket
  - Once TCP has sent all of the outstanding bytes…
  - … then TCP sends a FIN
- Receiving a FIN: EOF
  - Process is reading data from the socket
  - Eventually, the attempt to read returns an EOF

TCP State Diagram

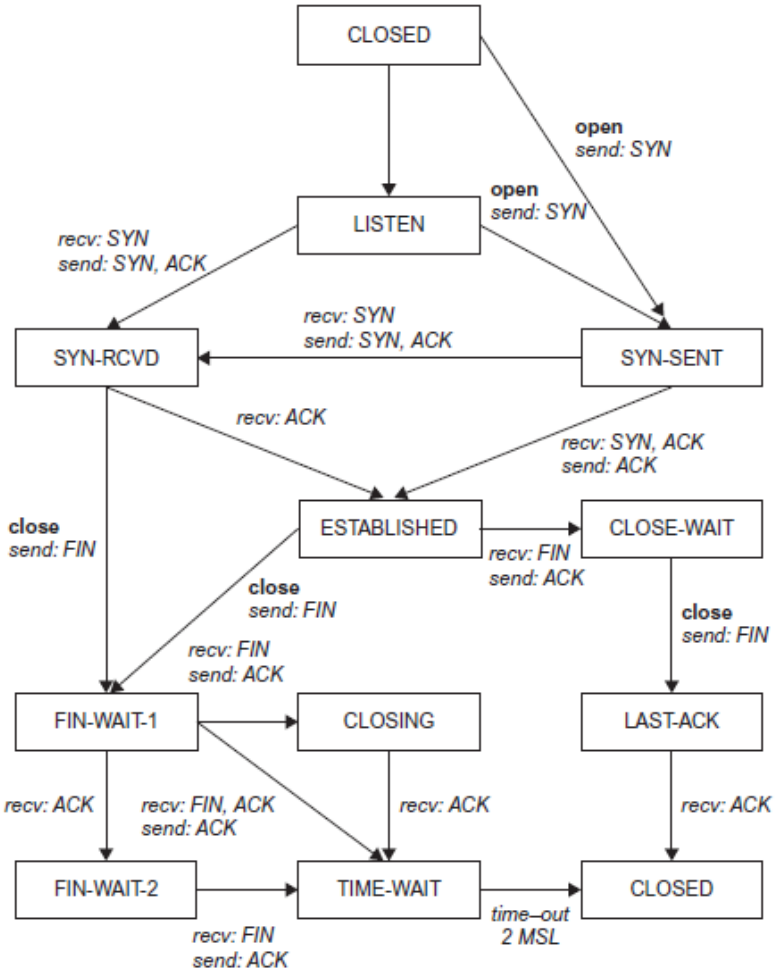


FIGURE 6.6

TCP state diagram.

UDP for Smart Objects



- Advantages
  - Very low overhead (header size and protocol logic) → low energy consumption; more room in each packet for application layer data; memory footprint
  - Simplicity of UDP fits well with several applications (e.g., temperature monitoring: losing a reading is not crucial) → in general, low reliability demands
  - Well suited to applications which require their own routing (application overlay mechanism)
  - UDP maps well to underlying multicast mechanisms
- Drawbacks
  - Loss of packets and no recovery mechanism → application has to recover (increase complexity)
  - Smart object networks have small packet sizes and UDP does not provide mechanisms to split their data → application layer decides proper packet size

TCP for Smart Objects

- Advantages
  - Smart object networks are typically lossy → reliable transport
  - TCP has problems in high-throughput applications over wireless links (not designed for wireless links), but high throughput is typically not relevant
  - Smart object networks interact with existing systems using TCP
  - TCP MSS (Maximum Segment Size) option is very useful (e.g., for 802.15.4 packets)
  - Core of TCP is simple (many of the complex mechanisms, e.g., sliding window and congestion control, aim at achieving high throughput)
  - Even though TCP header is larger than UDP header, it can be compressed
- Drawbacks
  - Reduced TCP implementations for smart objects do not achieve high throughput (as full-blown TCP stack): (i) no sliding window mechanism (only one TCP packet active at a time); (ii) delayed ACKs reduce throughput



UNIVERSITÀ
DI PARMA

IoT Lab

IPv6

Main sources

- Slides by Prof. Luca Veltri (see following slides)
- J. P. Vasseur and A. Dunkels, `Interconnecting Smart Objects with IP,` Morgan Kaufmann, 2010. Chapters 6 and 15.
- Ansar Pasha and Patrick Grossetete, “IPv6 @ Cisco”



UNIVERSITA' DEGLI STUDI DI PARMA
Dipartimento di Ingegneria dell'Informazione

IPv6

Luca Veltri

(mail.to: luca.veltri@unipr.it)

Course of Internet of Things, 2015/2016

IPv4

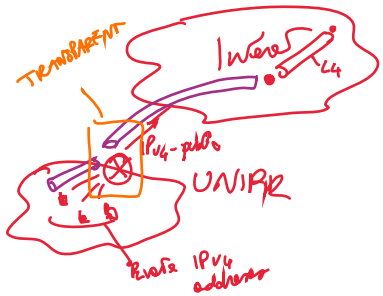
- RFC 791 del 1981
- More or less the same in 30 years
- Main issues:
  - **Limited addressing space**
  - **Growing of Routing Tables**
  - **Host configuration**
  - **Security**
  - **Quality of service**
  - **Mobility**

IPv4 issues: Addressing

- IPv4 addresses: 32-bit
  - **4,294,967,296 IPv4 "theoretical" addresses, but the exact number is far below**
- Depletion of IPv4 address
  - **early predictions (made at the time IPv6 was in its early phase of design, end of 90's): IPv4 address depletion as early as 2002**
- Mainly due to:
  - **growing countries (e.g. China)**
  - **Mobile phones**
  - **Always-on devices**
  - **IoT**

IPv4 issues: Addressing

- Several mitigation strategies were developed:
  - **better address allocation and subnetting**
  - **Classless inter-domain routing (CIDR)**
    - it also reduces the routing table sizes
  - **Network Address Translation (NAT)**
    - same public address shared by several hosts with private addresses
    - Some problems
      - introduce states in the network between end points
      - Some application protocols require specific application level gatways (ALG)
        - » e.g. FTP, SIP, DNS, ICMP
        - » need to maintain various protocol states
    - Incompatible with some security mechanism
      - e.g. IPSec



Internet Protocol v6

- IPv6 (RFC 2460) = the next generation Internet Protocol
  - **Complete redesign of IP addressing**
  - **Hierarchical 128-bit address with decoupled host identifier**
  - **Stateless auto-configuration**
  - **Simple routing and address management**
- Majority of traffic not yet IPv6 but..
  - **Most PC operating systems already have IPv6**
  - **Most routers already have IPv6 support**
  - **IoT is coming**

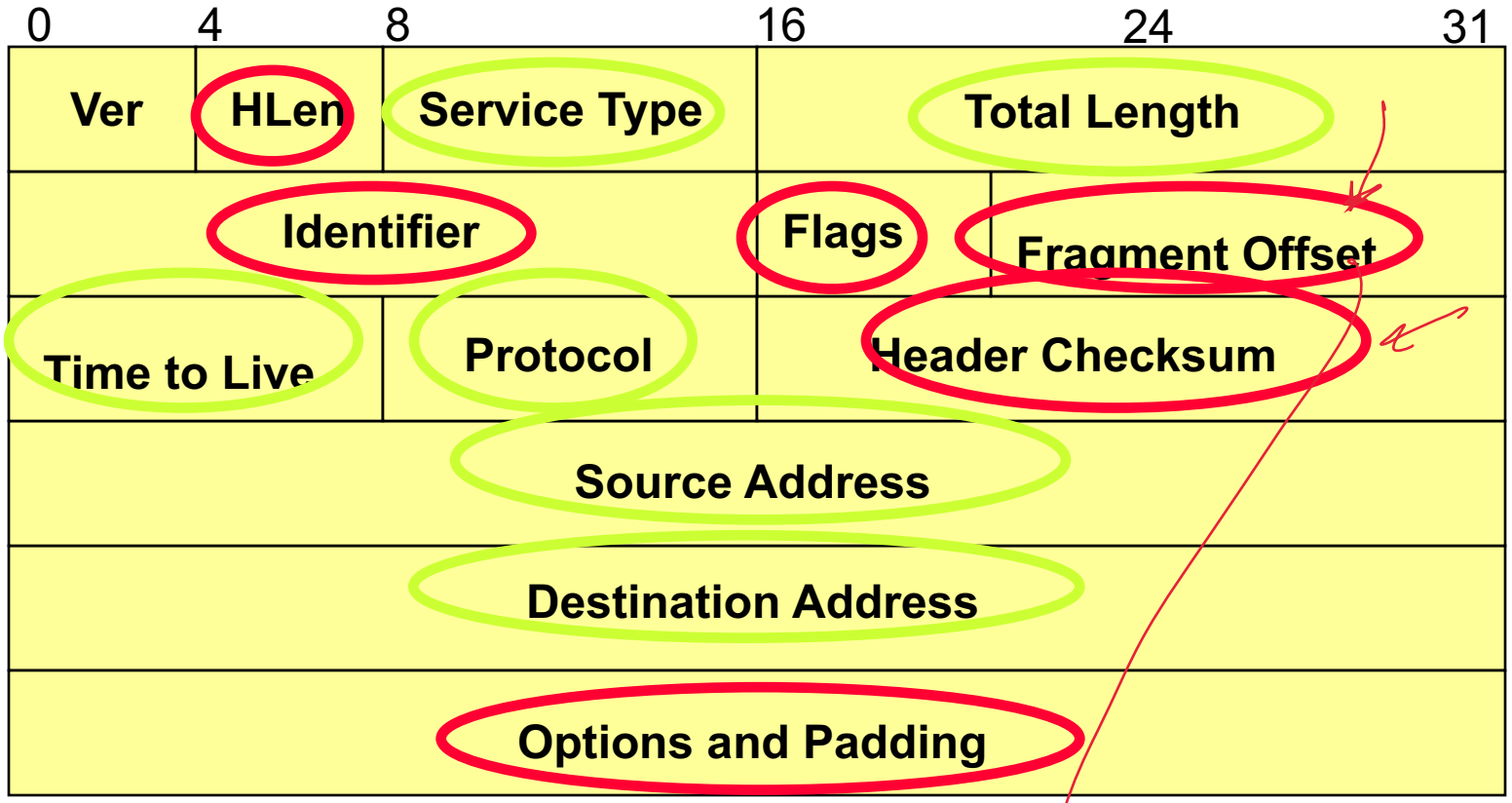
→ LEC. 6

IPv6 main characteristics

- Larger address space
  - **quadrupled to 16 bytes=128 bits**
  - **required for large-scale networks (urban networks, smart grids, industrial automation networks,..)**
- Larger number of levels of addressing hierarchy
  - **routing table efficiency**
- Autoconfiguration
  - **each node should be able to generate its own unique IPv6 address**
- Mobility support
  - **Mobile IPv6**
- Security
  - **IPSec/AH/ESP/IKE**

starting from its own unique MAC address (depending on the used interface)

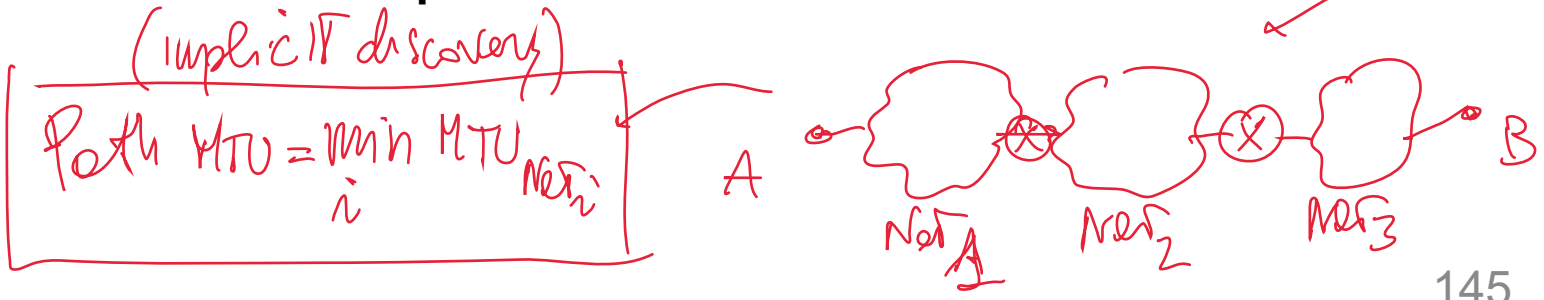
IPv4 vs IP6 Header format



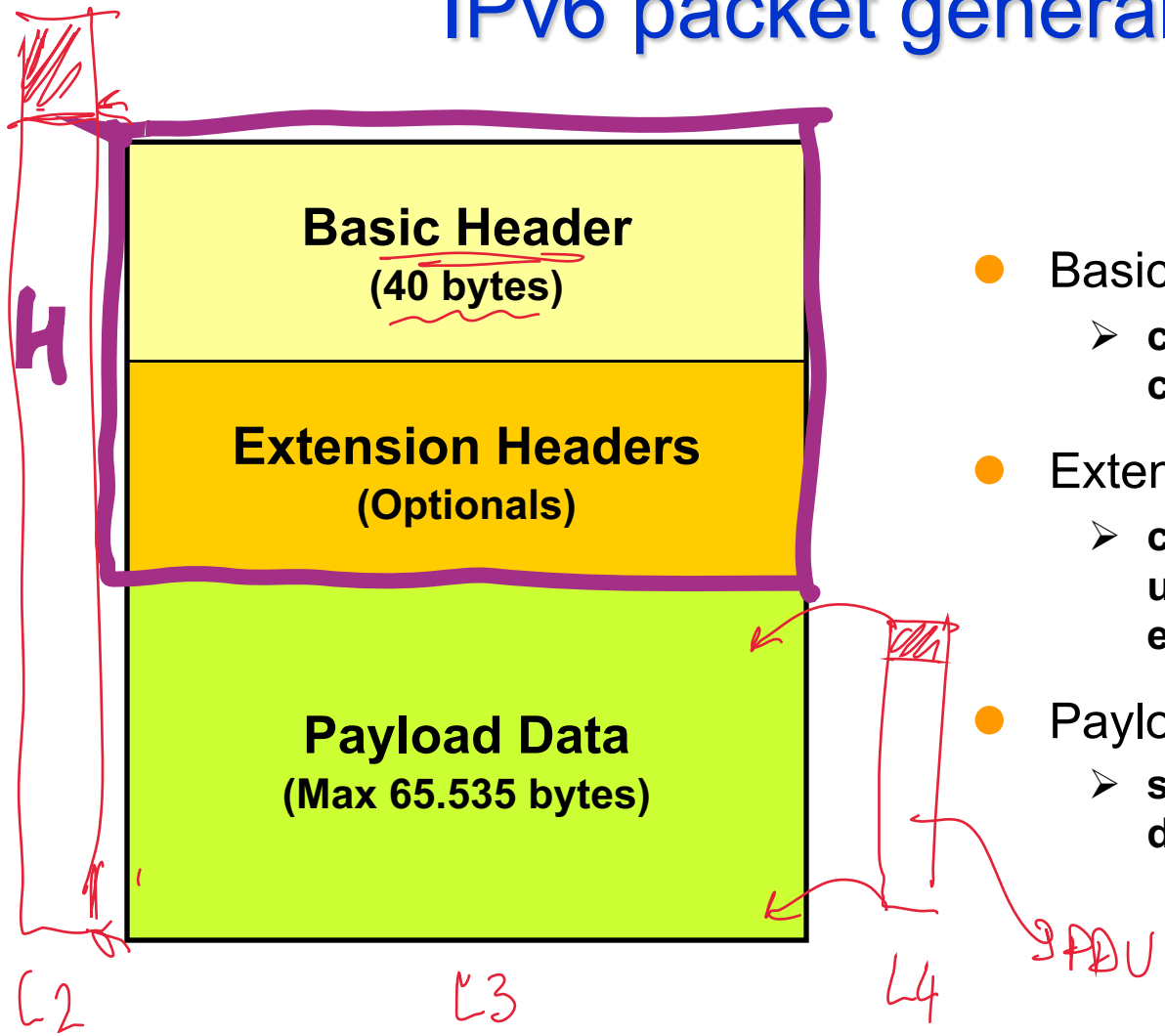
- 20 octects + options
- 13 mandatory fields
  - **one of them is also formed by 3 bits of flags**

IPv4 vs IP IP6 Header format

- No checksumming at the IP network layer → transport layer needs to take care
- No hop-by-hop segmentation: path MTU (Maximum Transmission Unit) discovery
- Header change
  - **Unused IPv4 header fields have been removed**
  - **Simpler structure with potentially daisy-chained headers**
  - **IPv6 header is twice as long (40 bytes) as IPv4 header without options (20 bytes)**
  - **64 bits aligned (various fields can be grouped in groups of 64 bits) → 64-bit processors!**

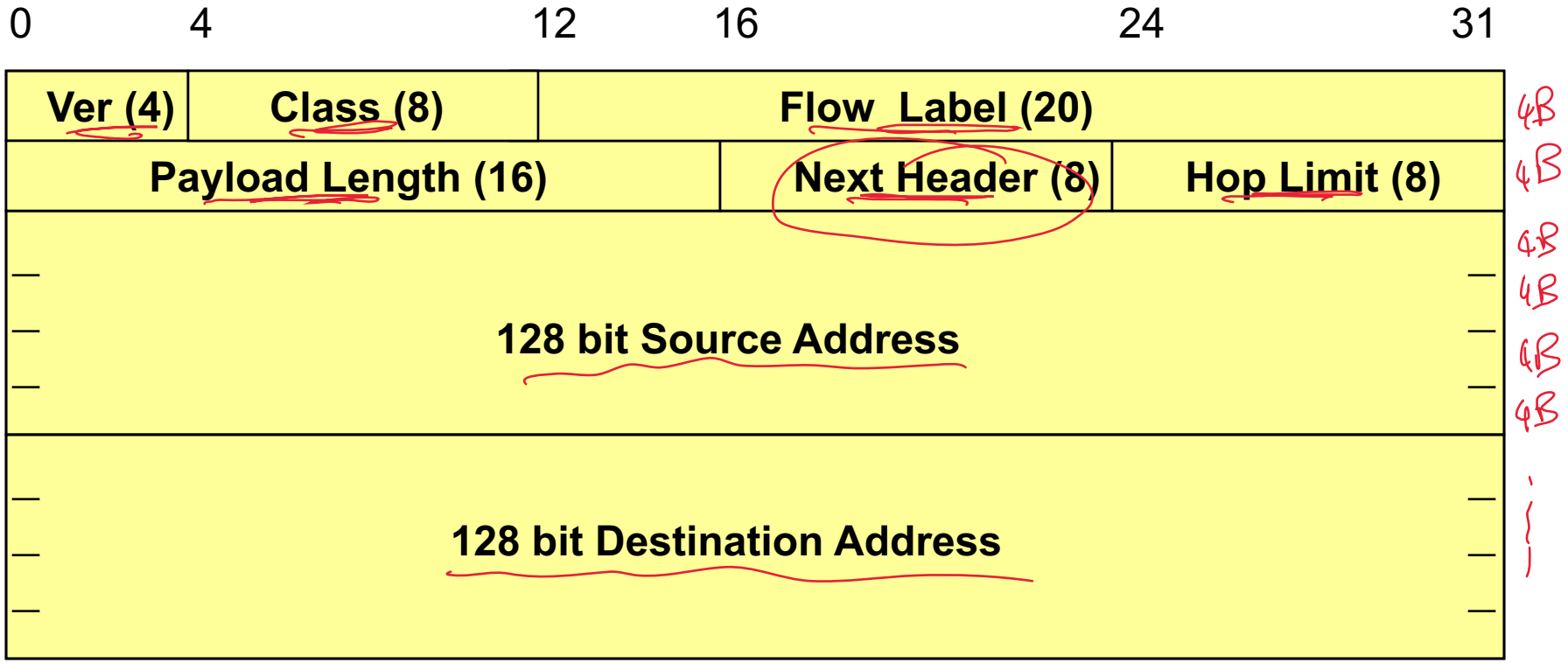


IPv6 packet general format



- Basic header
  - **contiene le informazioni comuni a tutti i datagrammi**
- Extension Headers
  - **contengono le opzioni utilizzate dai router intermedi e/o dall'host di destinazione**
- Payload Data
  - **sono i bit informativi elaborati dall'host di destinazione**

IPv6 Basic Header



40 octects
8 header fiels

IPv6 Basic Header

- Version (4 bits): IP versions number 6
- Traffic class (8 bits): used to indicate the Class of Service (CoS) of the packet
- Flow label (20 bits): a label may be used by a source node to refer to a sequence of packets identifying a flow that requires specific handling of the packet by routers along the path to its destination
- Payload length (16 bits): indicates the length of the payload (excluding the basic header)
- Next header (8 bits): identifies the header that follows the IPv6 packet header. It provides a flexible way to add optional extension headers
- Hop limit (8 bits): decremented each time the packet is forwarded by a node. When the hop limit field is equal to 0 (2^8 hops), the packet is discarded
- Source address: 128-bit IPv6 source address of the packet
- Destination address: 128-bit IPv6 destination address of the packet

IPv6 Basic Header

- Payload Length (16 bit)
  - **indicates the length of the payload (excluding the basic header)**
  - **normal maximum length is 65.535 byte**
  - **It is possible to extend it using "jumbo payload" (hop-by-hop options header)**
- Next Header (8 bit)
  - **identifies the header that follows the IPv6 packet header**
  - **Some values:**

0	Hop-by-hop options header	46	Resource Reservation Protocol
4	Internet Protocol	50	Encapsulating Security Payload
6	Transmission Control Protocol	51	Authentication Header
17	User Datagram Protocol	58	Internet Control Message Protocol
43	Routing Header	59	No Next Header
44	Fragment Header	60	Destination Options Header

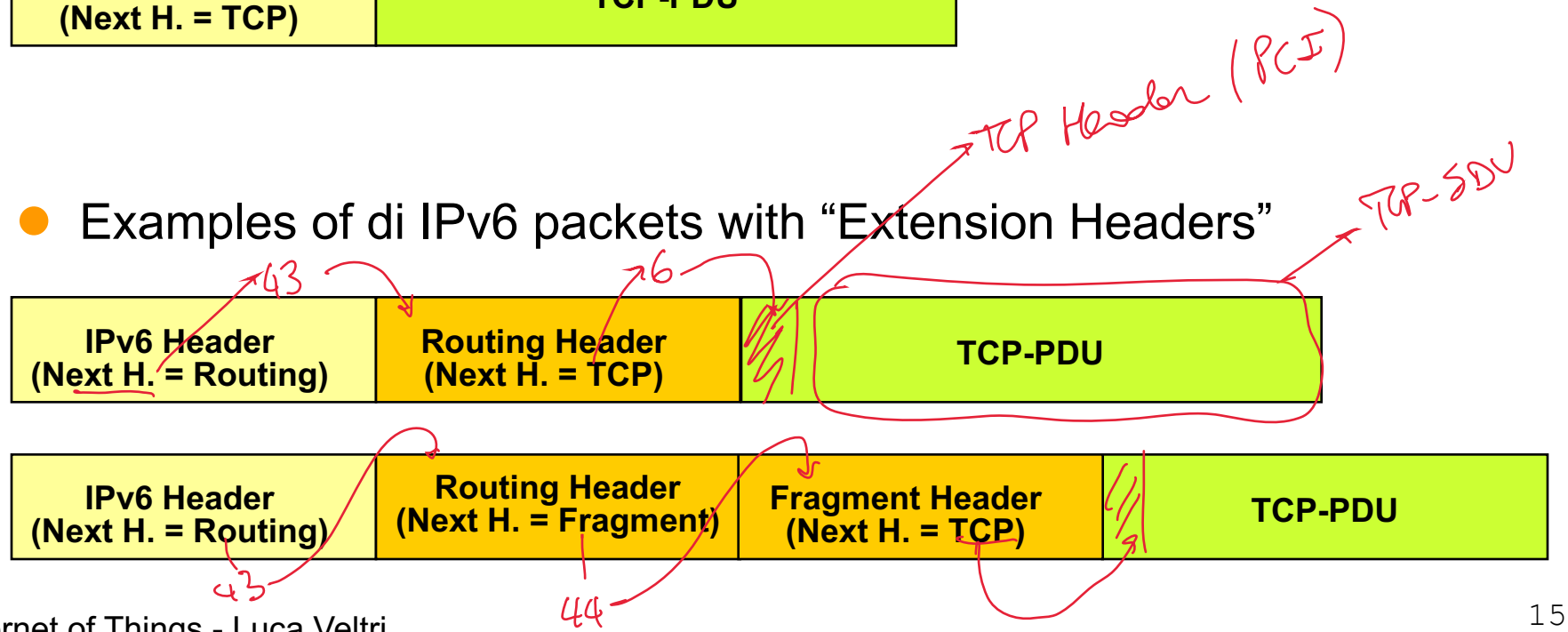
IPv6 Extension Headers

- Extension headers are inserted between the IPv6 basic header and the payload (upper layer data, e.g. TCP PDU)

- IPv6 "normal" packet

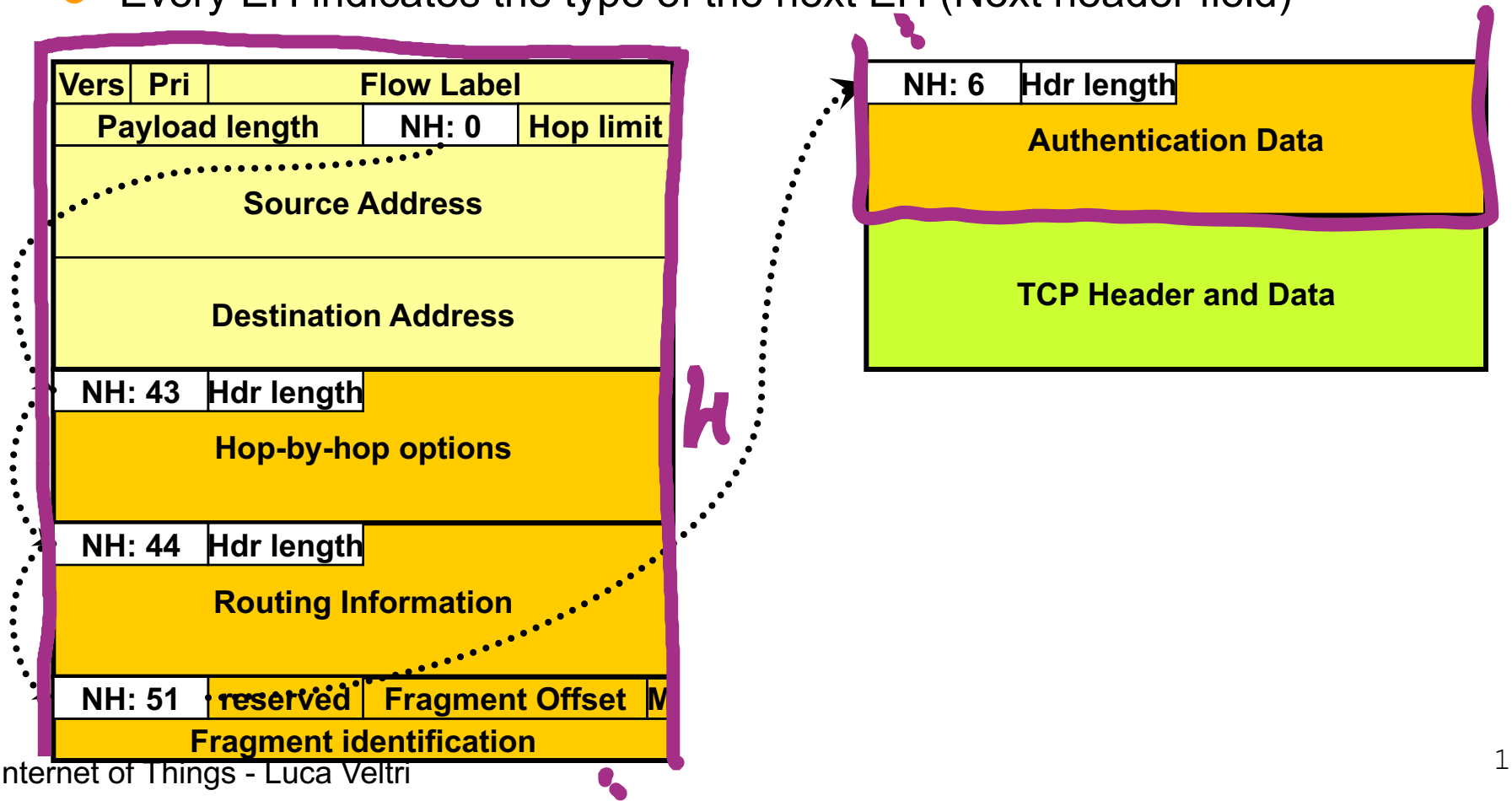


- Examples of di IPv6 packets with "Extension Headers"



IPv6 Extension Headers (cont.)

- Every EH has a length of 8*(1+n) byte, where *n* is the Header Extension Length value
- Every EH indicates the type of the next EH (Next header field)



IPv6 Extension Headers (cont.)

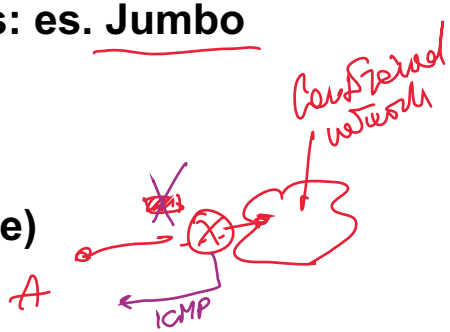
- Basic standard EH (in the right order):
  - **Hop-by-hop Options header**
  - **Destination Options header**
    - **only in prsence of Routing header**
  - **Routing header**
  - **Fragment header (only destination)**
  - **Authentication header (only destination)**
  - **Encapsulation security payload header (only destination)**
  - **Destination Options header (only destination)**
- Every EH may occur only 1 time
  - **The only exception is the Destination Options (at most 2 times)**

EHs relevant for intermediate nodes

EHs relevant only for the destination

IPv6 Extension Headers (cont.)

- Hop-by-hop Options (Type=0)
  - **It includes the options which involve all traversed routers: es. Jumbo Option (for packets with >65.575 bytes)**
- Routing (Type=43)
  - **It specifies a list of routers to be traversed (Strict o Loose)**
- Fragment (Type=44)
  - **It is useful to send fragmented packets (only from the source)**
  - **It a packets exceeds the link MTU (MTU min = 576 in IPv6!) it has to be discared and an ICMP msg is sent to the source, which can**
    - i) reduce the length of the subsequent packets
    - ii) send packets with the EH Fragment
- Destination Options (Type=60)
  - **It includes the options which need to be processed only at the destination node/s**
- Authetication (Type=51)
- Encapsulating Security Payload (ESP) (Type=50)



Hop-by-Hop Options

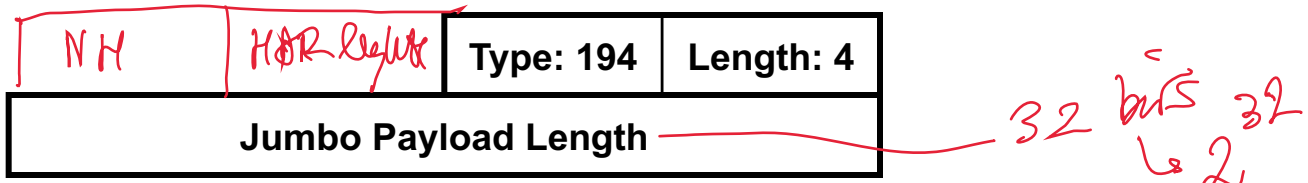
- It contains the options for each intermediate system over the path of the datagram
  - **It is processed also by intermediate nodes**
- It is constitute by a vsariable number of options codified as triplets TLV (Type-Length-Value)
  - **Type (8 bit): indicates the type of option**
  - **Length (8 bit): indicates the length of the field Value**
  - **Value: it transports the option and some indications for the router useful to process the option**
- Defined options:

Type	Option	Size	Allineamento
0	Pad 1	1 byte	nessuno
1	Pad N	2+n bytes	nessuno
194	Jumbo Payload Length	2+4 bytes	4·n + 2

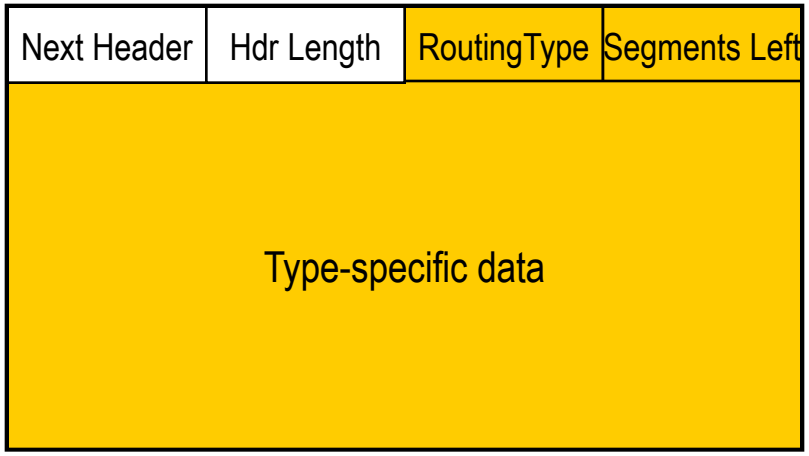
Hop-by-Hop Options Header

- Pad1 e PadN options
  - **Needed to insert filling octets in the header for alignment puroposes**
  - **Allow to increase the router processing speed**
    - Pad1: 1 byte: [type=0]
    - Pad2: 2+n bytes: [type=1 (1 byte), len (1 byte), N bytes]

- Jumbo Payload Length Option
  - **Allows to increase the maximum length of the datagram with respect to the value allowed by the basic header**
  - **If this option is used, the field payload length of the basic header has to be set to the value 0**

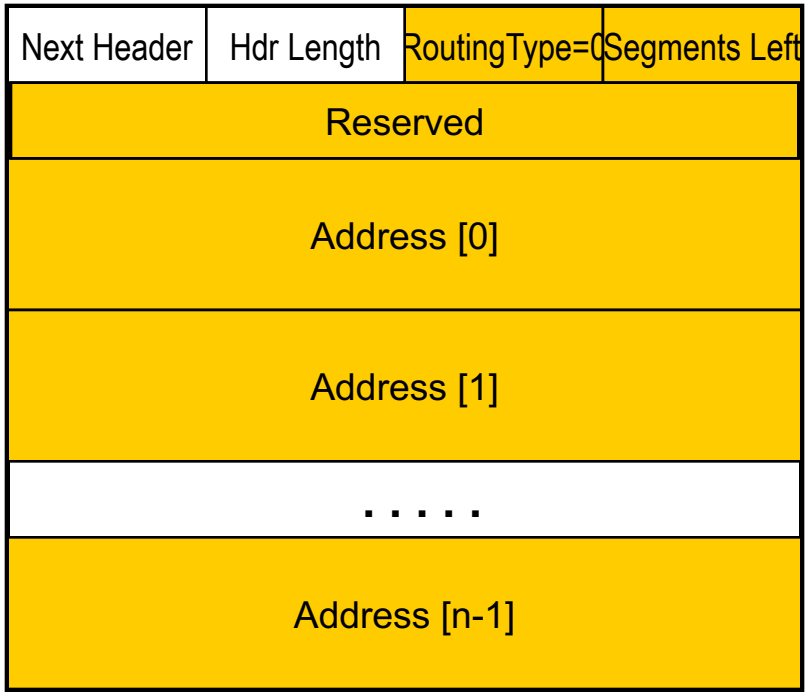


Routing Header



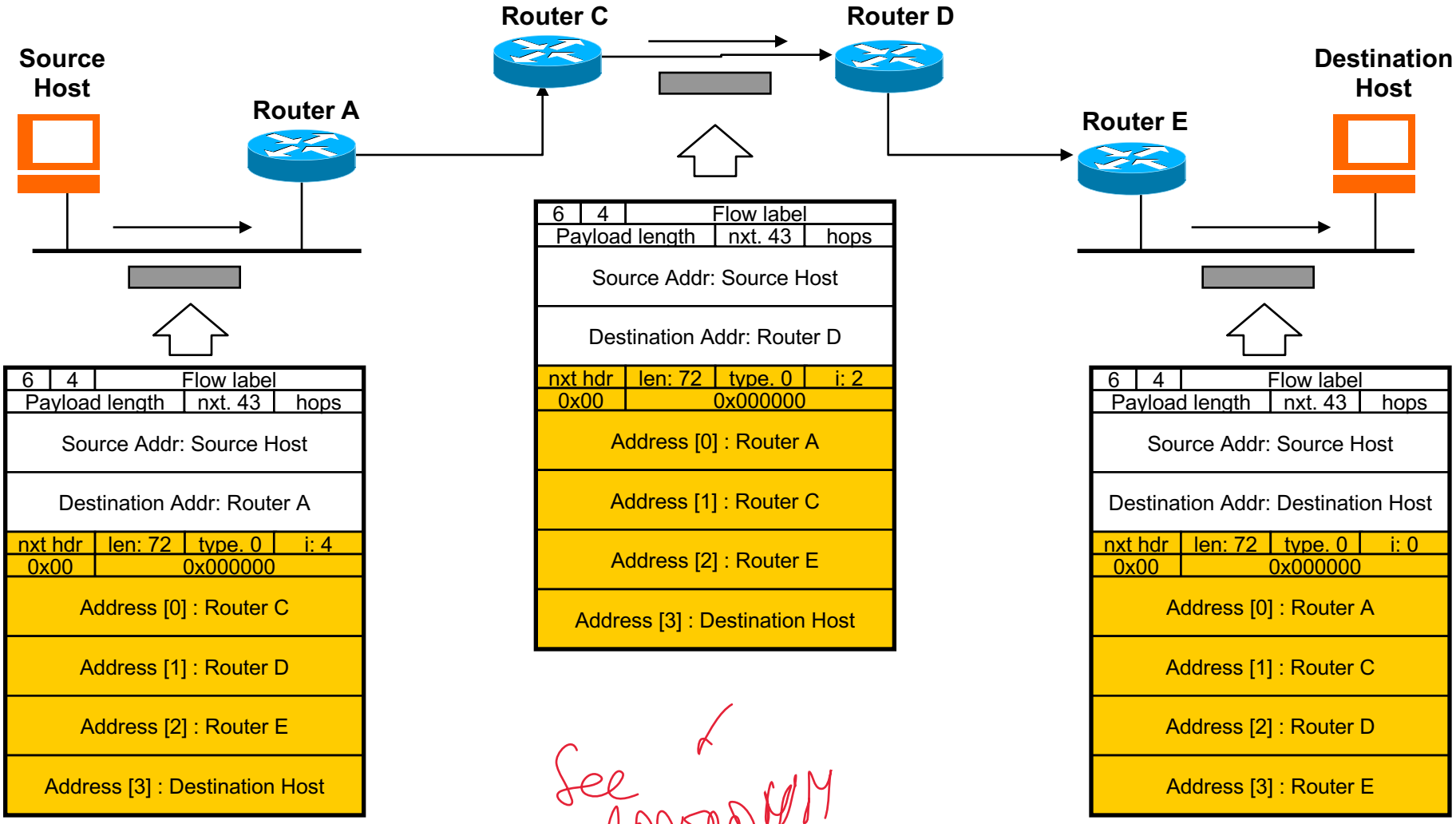
- Used by the source node to control the path followed by a packet
  - **similar to the Option *Loose Source* e *Record Route* of IPv4**
  - **processed only at the nodes with address in the *Dest Address***
- Format:
  - **Routing Type**
    - Identifies the type of routing
  - **Segments left**
    - It indicates the number of addresses which still need to be processed
    - Decremented by every node which processes it
  - **Type-specific data**
    - Field of variable length, multiple of 8 bytes
    - Depends on Routing Type

Routing Type 0 - Loose Source Route



- Address [0..n-1]
  - **Vector of *n* IPv6 addresses (*n* x 128 bit)**
  - **Adress[i] = (*i+1*)-th router along the path**
  - **At each hop, the index *i*=(*n-SegmentLeft*) is computed; the address *Address[i]* is exchanged with the *DA* (del basic Header), and the *Segment Left* field is decremented**
    - At the beginning:
      - DA= first node
      - Address[0]= second node
      - Address[n-1]= last nodo
    - At the end one has:
      - DA= last node
      - Address[0]= first node
      - Address[n-1]= before than last node

Loose Source Route - Example



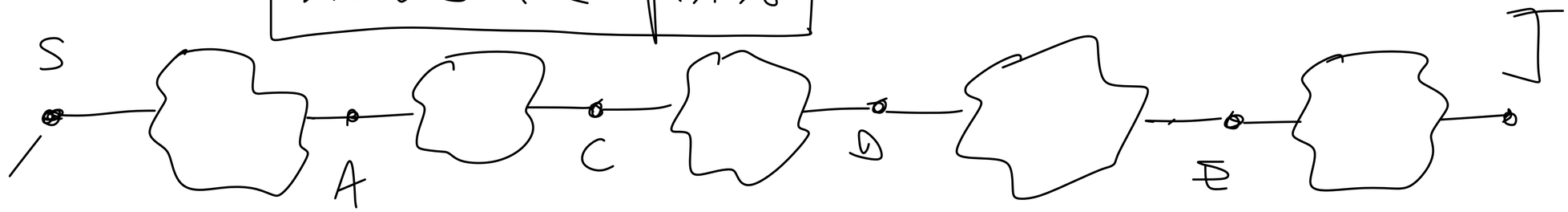
See Addendum

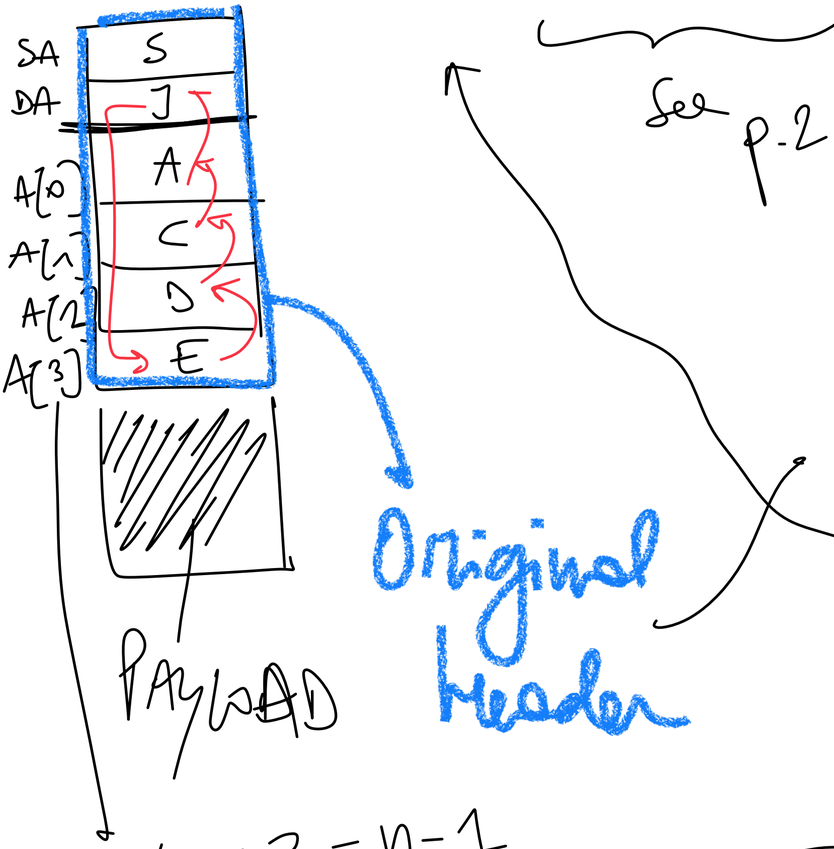
Lec. 6

IoT - Lecture 6 - March 12, 2026

S → SA (Source Address)
] → DA (Destination Address)

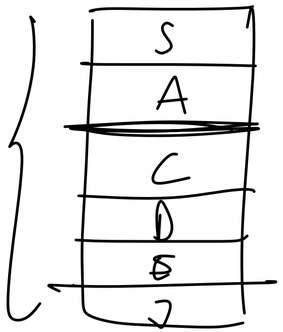
ADDENDUM @ p. 158





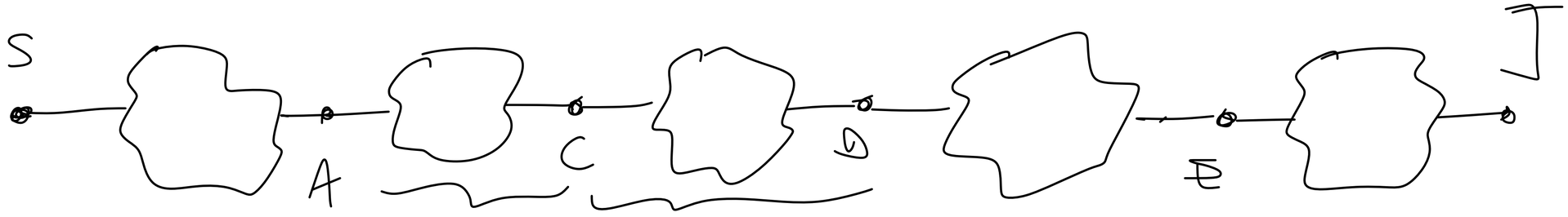
See p. 2

At Source @ S, the fields are changed as indicated →, obviously



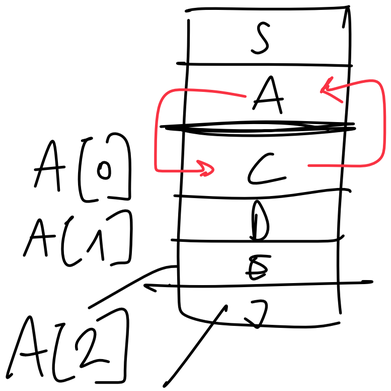
⇒ Pick flows between S and segments legs = 4 (fixed)

1



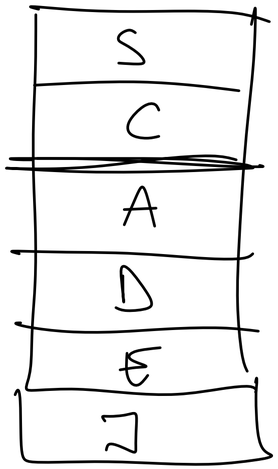
see p.3

Checks segmss legs = 4
then layer is DA
the address i = n − n = 0



they, if decrements
segmentslegs → 3

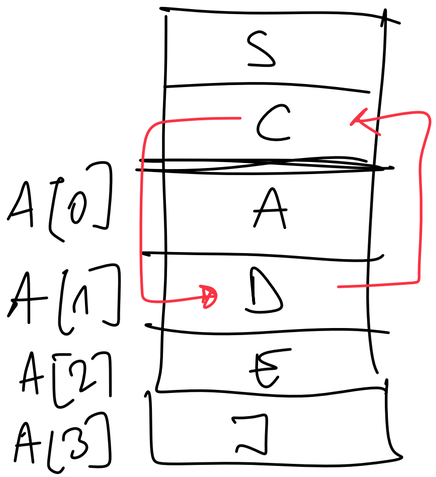
the packet formatted from A to C
is



2

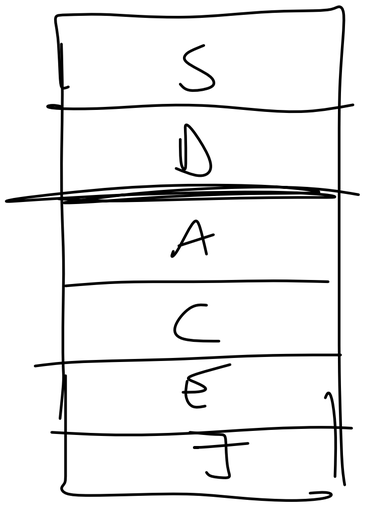
and segments legs = 3

Node C checks segments_left = 3 → it has to retransmit
→ it inserts as DA the address of v = 4 - 3 = 1 → D



and segments_left is set to 2

⇓ the packet transmitted to D is



w segments_left = 2

etc....

3